

I actually think your instinct is the right one.

Many people start by building the UI first, but for an AI production pipeline, **the backend is the product**. Once the backend is solid, you can build any frontend (Next.js, React, desktop app, CLI, etc.) on top of it.

I would make one important change

You said:

I will do the calling and orchestration by hand, so I understand the stages.

I would go one step further.

Don't build an orchestrator yet. Build a collection of independent services.

Think of your project as a toolbox.

```
graph TD; Narrative --> Script_Generator[Script Generator]; Script_Generator --> Audio_Generator[Audio Generator]; Audio_Generator --> Image_Generator[Image Generator]; Image_Generator --> Video_Generator[Video Generator]; Video_Generator --> Post_Processor[Post Processor];
```

Each one should be executable independently.

That will make debugging dramatically easier.

I would organize the backend like this

```
backend/
├── app/
└── api/
```

```
|
|
| └─ core/
|
|
| └─ models/
|
|
| └─ schemas/
|
|
| └─ services/
|
|
|     narrative/
|     script/
|     audio/
|     images/
|     video/
|     postproduction/
|
| └─ storage/
|
| └─ workflows/
|
| └─ utils/
|
└─ projects/
|
└─ tests/
|
└─ requirements.txt
```

Notice that **services** contain AI logic.

Not routes.

I wouldn't organize assets by type first

Instead, organize by project.

Example

```
projects/

  volcanoes/

    narrative/

      input/

        output/

      script/
```



```
    input/

    output/

audio/

    input/

    output/

images/

    input/

    output/

video/

    input/

    output/

post/

    input/

    output/
```

Each stage is completely isolated.

Even better

```
projects/

    volcanoes/

        001_introduction/

            audio/

            images/

            clips/

        002_magma/

            audio/
```



```
images/  
  
clips/  
  
003_eruption/  
  
audio/  
  
images/  
  
clips/
```

Later this will make rendering much easier.

Every service should follow the same interface

Example

```
AudioService  
  
generate()  
  
validate()  
  
save()  
  
load()  
  
status()
```



Image service

```
ImageService  
  
generate()  
  
validate()  
  
save()  
  
load()  
  
status()
```



Video service

```
VideoService
```



```
generate()
```

```
validate()
```

```
save()
```

```
load()
```

```
status()
```

Everything feels identical.

Don't tie yourself to OpenAI

Create an abstraction.

```
LLM
```



```
↓
```

```
OpenAI
```

```
Claude
```

```
Gemini
```

```
Ollama
```

```
Azure
```

```
OpenRouter
```

Example

```
class BaseLLM:
```



```
    generate()
```

```
    chat()
```

```
    stream()
```

Then

```
OpenAILLM(BaseLLM)
```



```
GeminiLLM(BaseLLM)
```

```
ClaudeLLM(BaseLLM)
```

Your services never know which model is being used.

Same for Image Models

```
ImageProvider
```



```
↓
```

```
Flux
```

```
Imagen
```

```
Ideogram
```

```
Midjourney
```

```
Recraft
```

```
Stable Diffusion
```

Same interface.

```
generate_image()
```



```
edit_image()
```

```
upscale()
```

Same for Audio

```
AudioProvider
```



```
↓
```

```
ElevenLabs
```

```
Google
```

Azure

OpenAI

Amazon Polly

Again

```
generate_voice()
```

```
clone_voice()
```

```
music()
```

```
sound_effect()
```



Project Folder

Imagine producing Episode 12.

```
Episode_12/
```

```
    metadata.json
```

```
    narrative/
```

```
    script/
```

```
    scenes/
```

```
        scene_001/
```

```
            prompts/
```

```
            audio/
```

```
            images/
```

```
            clips/
```

```
            renders/
```

```
        scene_002/
```

```
        scene_003/
```

```
    final/
```



Everything for one episode lives together.

Metadata

Don't rely only on filenames.

Create

metadata.json



Example

</> JSON



```
{
  "title": "Volcanoes",

  "status": "IMAGE_GENERATION",

  "language": "English",

  "voice": "Rachel",

  "image_model": "Flux",

  "video_model": "Veo",

  "current_scene": 5
}
```

Every service reads this.

Input / Output folders

I really like your idea.

I'd make it even more explicit.

audio/



input/

narration.txt

settings.json

```
output/  
  
  narration.wav  
  
  narration.mp3  
  
  metadata.json
```

Images

```
images/  
  
  input/  
  
    prompt.txt  
  
    style.json  
  
  output/  
  
    volcano.png  
  
    volcano.webp
```

Video

```
video/  
  
  input/  
  
    storyboard.json  
  
    image.png  
  
    narration.wav  
  
  output/  
  
    scene.mp4
```

Each module becomes almost "plug-and-play."

API

I wouldn't create endpoints like

```
/generateImage
```



Instead

```
POST
```



```
/projects/{id}/audio
```

```
POST
```



```
/projects/{id}/images
```

```
POST
```



```
/projects/{id}/script
```

```
POST
```



```
/projects/{id}/video
```

Everything revolves around the project.

One thing I strongly recommend

Because you'll be experimenting a lot with prompts, providers, and AI outputs, I would introduce a **Job** concept from the very beginning—even if you're triggering everything manually.

For example:

```
Project
```



```
|
├─ Narrative
├─ Script
├─ Audio
├─ Images
└─ Video
```

```
Each generation creates a Job.
```

```
Job #34
```

```
  Type: Image Generation
  Provider: Imagen
  Model: imagen-4
```

```
Status: Completed
Started: ...
Finished: ...
Input Folder: ...
Output Folder: ...
```

This gives you several advantages:

- You can rerun only failed jobs.
- You maintain a history of generations.
- You can compare outputs from different prompts or models.
- The future UI can display a complete activity log.
- If you later automate the pipeline, the orchestrator simply schedules jobs instead of directly invoking services.

In other words, your services remain focused on **doing work**, while jobs represent **work that has been requested and tracked**.

My roadmap for your first milestone

I would deliberately avoid databases and frontend initially. Instead, I'd aim for a backend that can process a complete episode from the command line or through simple API calls:

1. Build the project directory manager.
2. Implement a provider abstraction for LLMs, image generators, audio generators, and video generators.
3. Implement the Narrative, Script, Audio, Image, and Video services, each reading from `input/` and writing to `output/`.
4. Add a Job manager that records every execution and its metadata.
5. Expose each service through FastAPI endpoints so you can trigger them with tools like Swagger UI or Postman.

At that point, you'll have a robust AI production engine. Building the Next.js dashboard afterward becomes mostly a matter of visualizing projects, jobs, and assets rather than embedding business logic into the frontend. That's a much cleaner architecture and will make the application easier to extend as you add new AI providers and production stages.